Computer Vision: Exam Question Bank (57 questions) with Model Answers

Questions 1–19 are short answers (2 marks), 20–38 medium (5 marks), and 39–57 long (10 marks). The numbering matches the question list exactly. Each answer ends with → **notes:** pointing to the full explanation in this repository.

The GAN "notebook" questions (16–19, 30–32, 47, 48, 53) refer to the course's MNIST GAN: `week-4-gans/notes/01_gan_basics.md`.

Short answers (1–19)

1. Define a pixel in the context of digital images. A pixel (picture element) is the smallest unit of a digital image: one cell of the image grid, holding a number (grayscale intensity, 0 = black to 255 = white in 8-bit images) or a small vector of numbers (e.g. B, G, R). An image is a matrix of pixels addressed by row (y) and column (x), with the origin (0, 0) at the top-left. → notes: `week-1/notes/day1_images_and_colors.md`

2. What is the difference between grayscale and color image representation? A **grayscale** image is a 2-D matrix (height × width) with one intensity value per pixel. A **color** image is a 3-D array (height × width × 3) with three channel values per pixel (e.g. B, G, R), so it carries hue information but needs 3× the memory. → notes: `week-1/notes/day1_images_and_colors.md`

3. Why is OpenCV's color order BGR instead of RGB? It's historical. When OpenCV was created (around 2000), BGR was a common byte order in camera hardware and Windows image formats, and OpenCV kept it for backward compatibility. Practical consequence: convert with `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before displaying with Matplotlib, or red and blue appear swapped. → notes: `week-1/notes/day1_images_and_colors.md`

4. What does the `img.shape` attribute return in NumPy? A tuple of the array's dimensions: **(height, width, channels)** for a color image, e.g. (480, 640, 3), and **(height, width)** for a grayscale image. The order is rows first (height), then columns (width). → notes: `week-1/notes/day2_transformations.md`

5. Define a feature in computer vision. A feature is a distinctive, measurable piece of information in an image, such as an edge, corner, blob, texture pattern or color region, that helps identify, match or describe objects. Good features are repeatable under changes in viewpoint, lighting and scale. → notes: `week-2/notes/01_feature_detection_harris_sift_orb.md`

6. What is a keypoint? A keypoint is a specific, distinctive location in an image (e.g. a corner) that can be reliably found again in other images of the same scene. It usually stores position, scale and orientation, and is described by a **descriptor** vector used for matching. → notes: `week-2/notes/01_feature_detection_harris_sift_orb.md`

7. State one advantage of ORB over SIFT. ORB is much **faster** (it uses the FAST detector and binary BRIEF descriptors compared with Hamming distance), which suits real-time and mobile applications. It is also **free to use**: SIFT was patented until 2020. → notes: `week-2/notes/01_feature_detection_harris_sift_orb.md`

8. What does pooling achieve in CNNs? Pooling **downsamples** feature maps (e.g. 2×2 max pooling halves height and width). It reduces computation and parameters in later layers, enlarges the receptive field, and gives some **translation invariance** (a feature still counts even if it shifts slightly). → notes: `week-2-3/notes/01_building_blocks.md`

9. Define ReLU and explain its purpose. ReLU (Rectified Linear Unit): **f(x) = max(0, x)**. It adds non-linearity so the network can learn complex patterns. Its gradient is 1 for positive inputs, which reduces the vanishing-gradient problem of sigmoid/tanh, and it is cheap to compute, so it speeds up training. → notes: `week-2-3/notes/01_building_blocks.md`

10. What is dropout and why is it used? Dropout randomly switches off a fraction p of neurons (e.g. 50%) on each training step, and turns them all back on at test time. It prevents neurons from relying on each other (co-adaptation), acts like training an ensemble of thinner networks, and so **reduces overfitting**. → notes: `week-2-3/notes/01_building_blocks.md`

11. State one key contribution of LeNet. LeNet-5 (Yann LeCun, 1998) was among the first successful CNNs. It introduced the **conv → pool → fully-connected** pattern with weight sharing and was used commercially to read **handwritten digits on bank cheques** (trained on MNIST). → notes: `week-2-3/notes/02_classic_cnns_lenet_alexnet_vgg.md`

12. What does "compound scaling" mean in EfficientNet? Scaling a network's **depth, width and input resolution together** by fixed ratios (controlled by one coefficient φ), instead of enlarging only one dimension. This gives better accuracy for the same compute. → notes: `week-2-3/notes/03_modern_cnns_resnet_efficientnet.md`

13. Define viewpoint variation in computer vision. Viewpoint variation is the change in an object's appearance when it is seen from different angles or positions (front vs side vs top). The same object can produce very different pixel patterns, so a recogniser must identify it regardless of camera angle.

14. What is occlusion, and why is it challenging? Occlusion happens when part of an object is hidden behind another object (a pedestrian behind a car). It is challenging because the visible evidence is incomplete: key features may be missing, and detectors and trackers can miss or mislabel the object.

15. Define a GAN in simple terms. A Generative Adversarial Network is **two neural networks that compete**. A **Generator** creates fake data (e.g. images) from random noise, and a **Discriminator** tries to tell real data from fakes. As each improves against the other, the generator learns to produce realistic new samples. → notes: `week-4-gans/notes/01_gan_basics.md`

16. What dataset is used in the notebook? MNIST: 60,000 training images of handwritten digits (0–9), each 28 × 28 grayscale, loaded with `torchvision.datasets.MNIST` and normalised to [−1, 1]. → notes: `week-4-gans/notes/01_gan_basics.md`

17. What is the input to the Generator? A **random noise vector z** of 100 numbers drawn from a standard normal distribution, `torch.randn(batch_size, 100)`. All the variety in the generated images comes from z (the latent space).

18. What does the Discriminator output? A **single probability between 0 and 1** (via a Sigmoid) that the input image is **real**. Close to 1 means "real", close to 0 means "fake".

19. Why do we usually keep only the Generator after training? The goal is to **generate new data**, and only the Generator does that. The Discriminator was a training tool: its job was to provide a learning signal to the Generator. Once training ends, generation just needs noise → G(z).

Medium answers (20–38)

20. Explain the role of Hue, Saturation, and Value in color representation. HSV describes color the way humans think about it: - **Hue (H):** the color type, as an angle on the color wheel (0° red, 120° green, 240° blue; in OpenCV 0–179). - **Saturation (S):** the purity or intensity of the color, from grey (0) to vivid (255). - **Value (V):** brightness, from black (0) to full brightness (255).

Because **color (H) is separated from brightness (V)**, a red object stays roughly the same hue in sun or shadow. That makes HSV ideal for color-based detection with `cv2.inRange`. → notes: `week-1/notes/day1_images_and_colors.md`

21. What is the purpose of a kernel in image processing? A kernel (filter or mask) is a small matrix (e.g. 3×3) that slides over the image. At each position it multiplies its values with the pixels underneath and sums the products to form the output pixel. Different kernels perform different operations: - averaging or Gaussian kernels **blur** and remove noise; - sharpening kernels enhance detail; - Sobel/Laplacian kernels **detect edges**; - in CNNs, kernels are **learned** feature detectors. → notes: `week-1/notes/day3_filtering_and_convolution.md`

22. How does histogram equalization enhance image contrast? It **redistributes pixel intensities** so they spread over the full 0–255 range. Each intensity is mapped through the **cumulative distribution function (CDF)** of the histogram: new = round(CDF(old) × 255). Crowded intensity ranges get stretched apart, so a dull, low-contrast image (histogram squeezed into a narrow band) becomes clearer, with a roughly flat histogram. OpenCV: `cv2.equalizeHist(gray)`. **CLAHE** does it locally per tile, to avoid amplifying noise. → notes: `week-1/notes/day4_histograms_and_thresholding.md`

23. Explain the concept of descriptors with examples. A descriptor is a **numeric vector that describes the neighbourhood around a keypoint**, so the same point can be matched in another image. - **SIFT:** 128 numbers, histograms of gradient orientations in a 4×4 grid around the point. Robust to scale and rotation. - **ORB/BRIEF:** 256-bit binary strings from pixel-intensity comparisons. Very fast; matched with Hamming distance. - **HOG:** histograms of gradient directions over cells, used for whole-window description (pedestrians, faces).

Matching = finding descriptors with the smallest distance between two images. → notes: `week-2/notes/01_feature_detection_harris_sift_orb.md`

24. How does FLANN improve feature matching efficiency? Brute-force matching compares every descriptor with every other (O(N·M)), which is slow for thousands of features. **FLANN** (Fast Library for Approximate Nearest Neighbors) builds search structures such as **KD-trees** (for SIFT-type float descriptors) or **LSH** (for binary ORB descriptors) and finds **approximate** nearest neighbours much faster, with negligible accuracy loss. It's usually combined with **Lowe's ratio test** (keep a match only if best distance < 0.7–0.75 × second best) to discard ambiguous matches. → notes: `week-2/notes/02_image_matching_flann_logo_detector.md`

25. What is the role of RANSAC in robust estimation? RANSAC (Random Sample Consensus) fits a model, such as a **homography**, from data that contains many **outliers** (wrong matches). It repeats: 1. pick a minimal random sample (4 matches for a homography); 2. fit the model; 3. count **inliers** that agree within a threshold.

It keeps the model with the most inliers and refits it on all of them. Wrong matches therefore can't distort the result. Used in image stitching, logo/object localisation and pose estimation (`cv2.findHomography(..., cv2.RANSAC, 5.0)`). → notes: `week-2/notes/02_image_matching_flann_logo_detector.md`

26. Explain the concept of skip connections in ResNet. A skip (residual) connection **adds a block's input directly to its output: y = F(x) + x**. - The layers only need to learn the **residual** F(x) = desired − x. Learning "no change" is easy (F = 0), so extra layers can't make the network worse. - During backpropagation the gradient flows through the "+x" path unchanged, which **prevents vanishing gradients**.

This let ResNet train very deep networks (152 layers) and solved the **degradation problem** (deeper plain nets having higher training error). → notes: `week-2-3/notes/03_modern_cnns_resnet_efficientnet.md`

27. How does YOLO differ from R-CNN in object detection?

	R-CNN	YOLO
approach	**two-stage**: ~2,000 region proposals (selective search), then a CNN classifies each	**one-stage**: one CNN pass predicts boxes + classes for a grid over the whole image
speed	very slow (seconds per image)	real-time (tens of frames per second)
accuracy	high localization accuracy	slightly lower, weaker on small, crowded objects
context	sees regions in isolation	sees the whole image at once (fewer background errors)

→ notes: `week-2-3/notes/04_object_detection_rcnn_yolo_ssd.md`

28. What is the role of the encoder and decoder in U-Net? - **Encoder** (contracting path): convolution + pooling layers that **downsample**, capturing *what* is in the image (context and semantics) while losing spatial detail. - **Decoder** (expanding path): **upsampling** (transposed convolution) layers that restore the resolution to produce a **pixel-wise segmentation mask** (*where*). - **Skip connections** copy encoder feature maps to the matching decoder level, so fine boundaries are recovered precisely. That's why U-Net excels in medical image segmentation.

→ notes: `week-2-3/notes/05_segmentation_fcn_unet_mask_rcnn.md`

29. Explain how histogram equalization helps with illumination changes. Under dim or uneven lighting, intensities crowd into a narrow band: dark images cluster near 0, washed-out images near 255. Equalization spreads them across the full range using the CDF, which **normalises brightness and contrast**, so images of the same scene under different lighting look more alike. This makes edges, features and thresholds work more consistently. **CLAHE** (adaptive, per-tile, with a clip limit) handles **uneven** lighting better (e.g. a shadow over half a face), because global equalization uses one mapping for the whole image. → notes: `week-1/notes/day4_histograms_and_thresholding.md`

30. Explain why normalization of MNIST images is important for GAN training. The images are normalised from [0, 1] to **[−1, 1]** with `Normalize((0.5,), (0.5,))`, because: 1. **It matches the Generator's Tanh output range (−1 to 1).** If real and fake pixels lived in different ranges, the Discriminator could spot fakes just by their value range, and G would learn nothing useful. 2. **Zero-centred inputs give stabler, better-scaled gradients**, which matters a lot in the delicate two-player training.

31. Describe the two steps in each training round (update D, update G). Every mini-batch runs two steps: 1. **Update D** (learn to tell real from fake): - real images → D → BCE loss with target **1** (`labels_real`); - noise → G → fakes → D (with fakes **detached**, so G isn't updated) → BCE loss with target **0** (`labels_fake`); - `loss_D = loss_real + loss_fake`, then backprop and step `optimizer_D`. 2. **Update G** (learn to fool D): new fakes → D → BCE with target **1** (`labels_real`): G wants D to call its fakes real. Backprop through D into G and step **only** `optimizer_G`.

32. What is the role of random noise vectors in generating new images? The noise vector z (100 random numbers from N(0, I)) is the **seed** of each image. The Generator is a deterministic function, so **all the variety comes from z**: different z gives a different digit, and the same z always gives the same digit. During training, G learns to map the simple noise distribution onto the complex distribution of real images. After training, sampling new z gives endless new images. Nearby points in this **latent space** give similar images.

33. Describe the mathematical operation of convolution and its role in feature extraction. 2-D convolution slides a kernel K over the image I. At each position it multiplies and sums:

$$\mathbf{G(x, y) = \Sigma_i \Sigma_j I(x + i, y + j) \cdot K(i, j)}$$

(Strictly, convolution flips the kernel; CNNs use this unflipped version, called cross-correlation.) The output feature map is large where the image patch **resembles the kernel's pattern**. - With a Sobel kernel [−1 0 1; −2 0 2; −1 0 1] the output highlights **vertical edges**. - In CNNs, kernel values are **learned**: early layers learn edges and colors, middle layers textures and parts, deep layers whole objects. - **Weight sharing** (one kernel for the whole image) keeps the parameter count small and makes features **translation-equivariant**.

Output size = (W − K + 2P)/S + 1. → notes: `week-1/notes/day3_filtering_and_convolution.md` , `week-2-3/notes/01_building_blocks.md`

34. Describe the difference between supervised and unsupervised learning with CV examples. - **Supervised:** trained on **labelled** data (input → known answer) to predict labels for new inputs. CV examples: image classification (cat vs dog), object detection with labelled boxes (YOLO), segmentation with labelled masks (U-Net), digit recognition (MNIST with labels). - **Unsupervised: no labels**; it finds structure in the data. CV examples: **k-means clustering of SIFT descriptors** into visual words (Bag of Visual Words), image clustering, anomaly detection, and **GANs** learning to generate images from unlabelled data. - Key difference: supervised needs costly annotation and learns a mapping to known outputs; unsupervised needs no labels but its results are harder to evaluate.

→ notes: `week-2/notes/04_ml_in_cv_knn_svm.md`

35. Describe how Vision Transformers use attention to capture global context. - A ViT **splits the image into patches** (e.g. 16×16), flattens and linearly projects each into a token, adds **position embeddings**, and prepends a [CLS] token. - **Self-attention:** each patch makes a Query, Key and Value; attention weights = softmax($QK^T/\sqrt{d}$), and each patch's new representation is a weighted mix of **all** patches' Values. - So from the **first layer** any patch can relate to any other (e.g. a dog's head to its tail across the image), whereas a CNN's receptive field grows only gradually through many layers. - Multi-head attention captures several kinds of relationships in parallel. - ViTs need large datasets (or pre-training), because they have fewer built-in assumptions (inductive biases) than CNNs.

→ notes: `week-2-3/notes/07_attention_and_vision_transformers.md`

36. Discuss how multi-scale feature extraction addresses scale variation in CNNs. Objects appear at very different sizes (a near vs a far car), but a single feature map sees only one scale. Multi-scale methods look at several resolutions: - **Image pyramids:** run the detector on resized copies (as in classical Haar/HOG). - **Multi-scale feature maps: SSD** predicts from feature maps at several depths: early, high-resolution maps catch small objects; deep, coarse maps catch large ones. - **Feature Pyramid Networks (FPN):** combine deep semantic features with high-resolution shallow features through a top-down path plus lateral connections. - **Inception modules:** parallel 1×1, 3×3 and 5×5 filters in one layer. - **Anchor boxes** of multiple sizes and aspect ratios, and **dilated convolutions** (larger receptive field without downsampling).

Result: robust detection of both small and large objects. → notes: `week-2-3/notes/04_object_detection_rcnn_yolo_ssd.md`

37. Explain the minimax objective function of GANs.

$\min_G \max_D V(D, G) = E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p_z}[\log(1 - D(G(z)))]$

- **D maximises V:** it wants D(x) → 1 for real images (first term large) and D(G(z)) → 0 for fakes (second term large).
- **G minimises V:** it can only affect the second term, and wants D(G(z)) → 1, making log(1 − D(G(z))) very negative.
- It's a **zero-sum two-player game**: one player's gain is the other's loss.
- At the optimum, p_g = p_data and **D(x) = ½ everywhere** (a Nash equilibrium).
- In code it becomes binary cross-entropy: D trains with targets 1 (real) and 0 (fake); G trains with target 1 on its fakes (the non-saturating version, see Q57).

→ notes: `week-4-gans/notes/01_gan_basics.md`

38. Discuss the three common problems in GAN training: mode collapse, instability, and D too strong. 1. **Mode collapse:** G finds a few outputs that fool D and produces only those (e.g. every sample is the same "8"), with little diversity. *Fixes:* minibatch discrimination, unrolled GANs, Wasserstein loss, changing the optimizer settings (our run was fixed by Adam $\beta_1 = 0.5$). 2. **Instability / non-convergence:** the losses oscillate and never settle, because each network trains against a **moving target**. *Fixes:* lower learning rates, Adam($\beta_1 = 0.5$), DCGAN design rules (BatchNorm, LeakyReLU, strided convolutions), spectral normalisation. 3. **D too strong:** D separates real from fake almost perfectly, $D(G(z)) \approx 0$, so the original G loss $\log(1 - D(G(z)))$ **saturates** and G gets **vanishing gradients** and stops learning. *Fixes:* the non-saturating G loss, label smoothing (real target 0.9), adding noise to D's inputs, a smaller or less-trained D, WGAN.

Tip: losses alone don't show quality; watch a fixed-noise image grid every epoch. → notes: `week-4-gans/notes/02_gan_training_dynamics.md`

Long answers (39–57)

39. Explain the process of thresholding and its types with examples. Thresholding converts a grayscale image into a **binary** image (foreground and background) by comparing each pixel with a threshold T: pixel > T → 255 (white), else → 0 (black). It's the simplest form of segmentation: separating objects from the background before finding contours or measuring.

Types: 1. **Simple (global) thresholding:** one fixed T for the whole image. `cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)`. Variants: `THRESH_BINARY_INV` (inverted), `THRESH_TRUNC` (values above T are capped at T), `THRESH_TOZERO` (values below T become 0), `THRESH_TOZERO_INV`. *Example:* scanned black text on white paper. 2. **Otsu's thresholding:** chooses T **automatically** by maximising the between-class variance of the histogram. Best for **bimodal** histograms. `cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)`. *Example:* coins on a uniform table. 3. **Adaptive thresholding:** computes a **separate T for each neighbourhood** (mean or Gaussian-weighted mean of a block, minus a constant C), so it handles **uneven illumination**. `cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)`. *Example:* a document photographed with a shadow across it, or reading a whiteboard. 4. **Color (range) thresholding:** keep pixels inside a range per channel, usually in HSV: `cv2.inRange(hsv, lower, upper)`. *Example:* detecting a red ball or ripe fruit.

Pre-processing tips: blur first (Gaussian) to reduce noise; follow with morphology (opening/closing) to clean the mask. **Applications:** document binarization/OCR, medical imaging (separating tumours or bone in X-rays), industrial inspection, object counting, licence-plate reading. **Limitation:** global thresholds fail with uneven lighting or overlapping intensity distributions; use adaptive, Otsu or CLAHE then. → notes: `week-1/notes/day4_histograms_and_thresholding.md`

40. Discuss morphological operations (dilation, erosion, opening, closing) with diagrams and applications. Morphological operations process the **shape** of binary (or grayscale) images using a small **structuring element** (kernel), e.g. a 3×3 square, a cross or an ellipse.

- **Erosion:** a pixel stays white only if **all** pixels under the kernel are white. Objects **shrink**, thin bridges break, small white noise disappears.

- **Dilation:** a pixel becomes white if **any** pixel under the kernel is white. Objects **grow**, small holes and gaps fill, broken parts join.
- **Opening = erosion then dilation:** removes small **white specks** (salt noise) while keeping the main shapes' size.
- **Closing = dilation then erosion:** fills small **black holes** and gaps inside objects, and joins nearby parts.

Diagram (□ = background, ■ = foreground, 3×3 kernel):

```
 Original         Erosion          Dilation
□□□□□□□□         □□□□□□□□         □■■■■■□
□■■■■■□□         □□□□□□□□         ■■■■■■■
□■■■■■□□   →     □□■■■□□□   /     ■■■■■■■
□■■■■■□□         □□□□□□□□         ■■■■■■■
□□□□□□□□         □□□□□□□□         □■■■■■□

Opening: specks ■ around a shape vanish, the shape stays.
Closing: holes □ inside a shape fill in, the outline stays.
```

Others: morphological **gradient** (dilation − erosion = outline), **top-hat** (original − opening = small bright details).

Applications: - cleaning noisy scanned documents (opening removes specks, closing repairs broken letters); - separating touching objects before counting (erosion); - filling holes in segmentation masks (closing); - extracting boundaries (gradient); - medical imaging (cells, blood vessels); - industrial crack and defect detection; - fingerprint enhancement.

Structuring element size: a small kernel gives gentle changes; a large one removes more noise but can destroy details or merge objects. → notes: `week-1/notes/day5_morphological_operations.md`

41. Explain Harris Corner Detection: method, pros, cons, and applications. Idea: a **corner** is a point where shifting a small window in **any** direction changes the intensity a lot. On a flat region nothing changes; on an edge, change happens only across the edge.

Method: 1. Compute image gradients I_x and I_y (e.g. Sobel). 2. For each pixel, build the **structure tensor** M over a window: M = Σ w(x, y) [[I_x^2, I_xI_y], [I_xI_y, I_y²]]. 3. Compute the **corner response R = det(M) − k·(trace M)²**, with k ≈ 0.04–0.06 (det = $\lambda_1\lambda_2$, trace = $\lambda_1 + \lambda_2$ of M's eigenvalues). 4. Interpret: - R large and positive (both λ large) → **corner**; - R negative (one λ large) → **edge**; - |R| small → **flat** region. 5. Threshold R and apply **non-maximum suppression** to keep the strongest corners.

OpenCV: `cv2.cornerHarris(gray_float32, blockSize=2, ksize=3, k=0.04)`.

Pros: simple and fast; **rotation-invariant** (eigenvalues don't change with rotation); fairly robust to small illumination changes and noise; accurate corner localisation. **Cons: not scale-invariant** (a corner at one scale looks like an edge when zoomed in); sensitive to the choice of k, window size and threshold; produces **no descriptor**, so matching needs another method; struggles with noise and blur.

Applications: image stitching (panoramas), motion tracking and optical flow (Shi–Tomasi "good features to track" is a variant), camera calibration (chessboard corners), 3-D reconstruction, object recognition. → notes: `week-2/notes/01_feature_detection_harris_sift_orb.md`

42. Discuss the role of convolution, pooling, and activation functions in CNNs with examples. A CNN repeats **conv → activation → pool** blocks, then flattens into fully-connected layers that output class scores.

1. **Convolution layers: feature extraction.**
2. Learned kernels slide over the input and produce **feature maps** that respond to specific patterns.
3. **Weight sharing** (the same kernel everywhere) needs far fewer parameters than fully-connected layers and finds a pattern anywhere in the image.
4. *Example:* 32 filters of 3×3 on a 28×28 MNIST image produce 32 feature maps detecting edges at different angles. Deeper layers combine them: edges → corners and textures → digit parts → whole digits.
5. **Activation functions: non-linearity.**
6. Without them, stacked convolutions collapse into one linear operation.
7. **ReLU** f(x) = max(0, x) is the default: fast, sparse, and it reduces vanishing gradients.
8. *Example:* after convolution, ReLU keeps only positive edge responses. **Softmax** at the output turns scores into class probabilities (digit 7: 0.92).
9. **Pooling layers: downsampling.**
10. **Max pooling** (2×2, stride 2) keeps the strongest response in each window and halves the height and width.
11. It reduces computation and parameters, enlarges the receptive field, and adds **translation invariance** (a slightly shifted edge still gives the same max).
12. *Example:* 28×28 → 14×14 → 7×7 in LeNet-style nets. **Global average pooling** replaces large fully-connected layers in modern nets.

Together: convolution finds patterns, activation lets them combine non-linearly, pooling summarises and compacts them. Stacking the blocks builds a hierarchy from pixels to concepts. → notes: `week-2-3/notes/01_building_blocks.md`, `week-2/notes/08_cnn_basics.md`

43. Explain the evolution of CNN architectures from LeNet to VGG, highlighting their key innovations. 1. **LeNet-5 (1998, Yann LeCun)** - Small: ~60K parameters, 7 layers, input 32×32 grayscale; conv → pool (subsampling) → conv → pool → fully-connected; tanh/sigmoid activations. - **Innovations:** proved that **learned convolutional features + weight sharing + pooling** beat hand-crafted features, trained end-to-end with backpropagation. - Used for **handwritten digits on bank cheques** (MNIST). - *Limits:* small data and compute, so it couldn't scale to complex images. 2. **AlexNet (2012, Krizhevsky, Sutskever, Hinton)** - 8 layers (5 conv + 3 fully-connected), ~60M parameters, 224×224 RGB input. - **Won ImageNet 2012**, cutting the top-5 error from ~26% to **15.3%** and starting the deep-learning era. - **Innovations: ReLU** (training ~6× faster than tanh), **dropout** in the fully-connected layers (less overfitting), **GPU training** (2 GPUs), **data augmentation** (crops, flips, color jitter), overlapping max pooling, local response normalisation. 3. **VGG (2014, Oxford Visual Geometry Group)** - 16–19 layers (VGG-16, VGG-19), ~138M parameters. - **Innovation: simplicity and depth**: only **3×3 convolutions** (stride 1) and 2×2 max pooling, stacked deep. Two 3×3 convs see 5×5 (three see 7×7) with fewer parameters and more ReLU non-linearities than one large filter. - Uniform design; showed that **depth matters**; 7.3% top-5 error on ImageNet. - *Limits:* very heavy (memory and compute) and hard to train deeper, which ResNet later solved with skip connections.

Trend: deeper networks; ReLU replacing sigmoid; regularisation (dropout, augmentation); GPUs and big data (ImageNet); smaller, stacked filters. → notes: `week-2-3/notes/02_classic_cnns_lenet_alexnet_vgg.md`

44. Discuss the importance of ReLU and Dropout in training deep networks. ReLU, f(x) = max(0, x): - **Fixes vanishing gradients:** sigmoid's slope is at most 0.25 and near 0 when saturated, so gradients shrink layer after layer. ReLU's slope is exactly **1** for positive inputs, so gradients pass through deep stacks. - **Faster training:** a simple max operation with no exponentials (AlexNet trained ~6× faster than with tanh). - **Sparsity:** negative inputs become exactly 0, so only some neurons fire, which gives efficient, more separable representations. - **Caveat: dying ReLU.** Neurons stuck with negative input get zero gradient forever. Fixes: Leaky ReLU, PReLU, a sensible learning rate and He initialisation.

Dropout: - During training, each neuron is switched off with probability p (e.g. 0.5); at test time all neurons are active (outputs scaled accordingly). - **Prevents co-adaptation:** neurons can't rely on specific partners, so they learn robust, independent features. - **Acts like an ensemble** of many thinner sub-networks, averaged at test time. - **Reduces overfitting**, the main problem of large networks on limited data (AlexNet's fully-connected layers overfit badly without it). - *Costs:* slower convergence; p must be tuned; usually placed in fully-connected layers (BatchNorm reduces the need in conv layers).

Together: ReLU makes deep networks **trainable**; dropout makes them **generalise**. → notes: `week-2-3/notes/01_building_blocks.md`, `../deep-learning-genai/notes/04_activation_functions.md`, `../deep-learning-genai/notes/05_normalization_and_regularization.md`

45. Describe the four core challenges in computer vision (viewpoint, illumination, occlusion, scale) with examples. 1. **Viewpoint variation:** the same object looks different from different angles, because its 3-D shape projects to different 2-D pixel patterns. *Example:* a car seen from the front, side and top; a face in profile vs frontal (frontal Haar face detectors miss profiles). 2. **Illumination changes:** lighting changes pixel values without changing the object: shadows, overexposure, night vs day, colored light. *Example:* face recognition failing under harsh side light; a red traffic light looking orange or washed-out in sunlight; a global threshold failing on a half-shadowed document. 3. **Occlusion:** part of the object is hidden by another object or the frame edge, so features are missing. *Example:* a pedestrian behind a parked car (critical for self-driving cars); a tracked player hidden behind another; a face covered by a mask. 4. **Scale variation:** objects appear at very different sizes depending on distance. A detector tuned for one size misses others. *Example:* a nearby car fills the frame while a distant one is a few pixels; small objects in aerial or satellite images.

Why they're hard: in all four, the **object stays the same but the pixels change a lot**, so a vision system must be **invariant** to these changes while still telling different objects apart. (Other challenges: intra-class variation, background clutter, deformation, motion blur.) Techniques that overcome them: see Q52.

46. Describe the analogy of the Generator as a forger and the Discriminator as an inspector. - **The Generator is an art forger.** It starts with a blank idea (random noise z) and tries to paint convincing fake banknotes or paintings. At first its fakes are crude. - **The Discriminator is a police inspector / art expert.** It examines items and decides "genuine" (real training image) or "forgery" (generated image). - **The contest:** the inspector studies real notes and the forger's fakes and gets better at spotting flaws (training D). The forger learns from **which fakes got caught and why** (the gradient through D) and improves its technique (training G). - **Arms race:** as the inspector sharpens, the forger must improve, and vice versa. Each pushes the other. - **The ideal ending:** the forgeries become **indistinguishable** from real ones, and the inspector can only guess (50/50, D = 0.5): the Nash equilibrium. - **Mapping to code:** forger's idea = `torch.randn(batch, 100)`; forgery = G(z); inspector's verdict = D(x) ∈ [0, 1]; "inspector training" = D's step with labels 1/0; "forger improves" = G's step with target 1 (it wants to be called real). - **Limits of the analogy:** the forger never sees real notes directly, only the inspector's reactions; and a forger who finds one trick that always works and repeats it is **mode collapse**.

→ notes: `week-4-gans/notes/01_gan_basics.md`

47. Explain the architecture of the Generator in the notebook (layers, activations, output).

```
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(100, 256),   # noise → hidden
            nn.ReLU(),
            nn.Linear(256, 784),   # hidden → 28×28 pixels
            nn.Tanh())
    def forward(self, x):
        return self.fc(x).view(-1, 1, 28, 28)
```

- **Input:** a noise vector z of **100** values from N(0, 1) (`torch.randn(batch_size, 100)`).
- **Layer 1:** fully-connected **100 → 256**, then **ReLU** (non-linearity, so the mapping from noise to image can be complex).
- **Layer 2:** fully-connected **256 → 784** (one value per pixel of a 28 × 28 image).
- **Output activation: Tanh**, squashing each pixel to **[−1, 1]**, exactly the range of the normalised real MNIST images, so D can't tell fakes by their value range.
- **Reshape:** 784 values → (batch, 1, 28, 28), a one-channel grayscale image.
- **Parameters:** 100·256 + 256 + 256·784 + 784 = **227,344**.
- A simple MLP GAN; DCGAN replaces the linear layers with transposed convolutions + BatchNorm for sharper images.
- Trained with Adam (lr 0.0002) and BCE loss, using `labels_real` as the target for its fakes.

48. Explain the architecture of the Discriminator in the notebook (layers, activations, output).

```
class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(784, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid())
    def forward(self, x):
        return self.fc(x.view(-1, 784))
```

- **Input:** an image (real or fake), 1 × 28 × 28, **flattened to 784** values.
- **Layer 1:** fully-connected **784 → 256**, then **LeakyReLU(0.2)**. For negative inputs it passes a small slope (0.2 × x) instead of 0. This matters because **G learns only from gradients flowing back through D**: a plain ReLU could block them ("dead" units).
- **Layer 2:** fully-connected **256 → 1**.
- **Output activation: Sigmoid**, a single probability in [0, 1] that the image is **real**.
- **Parameters:** 784·256 + 256 + 256 + 1 = **201,217**.
- **Mirror of G:** G goes 100 → 256 → 784; D goes 784 → 256 → 1.
- Trained with **BCELoss**, targets 1 for real and 0 for fake, Adam lr 0.0002. Fakes are `.detach()` ed during D's step so G isn't updated then.

49. Compare RGB and HSV color models: advantages, limitations, and use cases in computer vision.

	RGB (OpenCV stores BGR)	HSV
components	Red, Green, Blue intensities (0–255 each)	**Hue** (color type, 0–179 in OpenCV), **Saturation** (purity), **Value** (brightness)
model	additive mixing of primary lights (how screens and cameras work)	cylindrical; separates color from brightness (closer to human perception)
advantages	native format of cameras, displays and files (no conversion); simple; what CNNs are trained on	**color is separated from lighting**, so detection is robust to shadows/brightness; intuitive ranges ("all reds")

	RGB (OpenCV stores BGR)	HSV
limitations	brightness is mixed into all three channels, so a shadow changes R, G **and** B and color thresholds break under lighting changes; not intuitive	conversion cost; **hue wraps around** (red is at both 0 and 179, so it needs two ranges); hue is unstable for very dark or grey pixels (low S or V)
use cases	display, storage, deep-learning input, general processing	**color-based segmentation and tracking**: skin detection, traffic lights, ripe-fruit sorting, ball tracking, green-screen removal

Rule of thumb: use RGB for storage, display and CNN input; convert to HSV (`cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`) when you need to **find objects by color** under varying light, using `cv2.inRange` on H, with minimum S and V to ignore greys and shadows. → notes: `week-1/notes/day1_images_and_colors.md`

50. Compare Haar cascades and HOG for face detection: methodology, strengths, weaknesses, and applications. Haar cascades (Viola–Jones, 2001): - **Method:** - **Haar-like features**: differences of pixel sums in adjacent black/white rectangles (e.g. the eye region is darker than the cheeks), computed instantly with an **integral image**; - **AdaBoost** selects the few most useful features out of ~160,000; - a **cascade** of increasingly complex stages rejects non-face windows early; - the window slides over an image pyramid. - **Strengths: very fast** (real-time on old CPUs); cheap; built into OpenCV (`cv2.CascadeClassifier`). - **Weaknesses:** works mainly on **frontal, upright faces**; sensitive to lighting, rotation and occlusion; many false positives; needs `scaleFactor` / `minNeighbors` tuning. - **Applications:** camera autofocus and face boxes, early webcam apps, simple real-time detection on low-power devices.

HOG (Histogram of Oriented Gradients, Dalal & Triggs, 2005) + linear SVM: - **Method:** compute gradients → split the window into cells (e.g. 8×8) → build a **histogram of gradient orientations** (9 bins) per cell → normalise over blocks (robust to lighting) → concatenate into a feature vector → classify with a **linear SVM** in a sliding window over an image pyramid (dlib's face detector uses HOG). - **Strengths:** captures **shape/edge structure** robustly; more accurate than Haar with fewer false positives; robust to illumination and small deformations. - **Weaknesses:** slower than Haar; still struggles with large pose changes, occlusion and very small faces; fixed window aspect ratio. - **Applications:** pedestrian detection (its original use), face detection (dlib), vehicle detection, people counting.

Summary: Haar = speed; HOG = accuracy and robustness. Both are now largely replaced by CNN detectors (MTCNN, RetinaFace, YOLO), which handle pose and occlusion far better. → notes: `week-2/notes/06_face_detection_haar_hog.md`

51. Compare R-CNN, YOLO, and SSD architectures: processes, strengths, weaknesses, and ideal applications. R-CNN family (two-stage): - **Process:** 1. **Selective search** proposes ~2,000 candidate regions; 2. each region is warped and passed through a CNN for features; 3. SVMs classify each region, and regression refines the box. - **Fast R-CNN** runs the CNN once per image and uses RoI pooling. **Faster R-CNN** replaces selective search with a learned **Region Proposal Network (RPN)**. - **Strengths: high accuracy and precise localization**, good on small objects (Faster R-CNN). - **Weaknesses: slow** (original R-CNN ~47 s per image; Faster R-CNN ~5–7 fps); complex multi-stage training. - **Ideal for:** accuracy-critical, offline tasks: medical imaging, satellite analysis, detailed inspection (Mask R-CNN adds segmentation masks).

YOLO, "You Only Look Once" (one-stage): - **Process:** divide the image into an **S×S grid**; in **one forward pass**, each cell predicts bounding boxes, confidence scores and class probabilities; then non-maximum suppression. - **Strengths: real-time** (45+ fps); sees the whole image, so fewer background false positives; simple end-to-end training. - **Weaknesses:** less precise localization; struggles with **small or crowded objects** (limited boxes per cell in early versions). - **Ideal for:** real-time video: autonomous driving, surveillance, robotics, drones, sports tracking.

SSD, Single Shot MultiBox Detector (one-stage): - **Process:** a single pass; default (anchor) boxes of several sizes and aspect ratios; predictions made from **multiple feature maps at different scales** (early high-resolution maps → small objects, deep maps → large objects). - **Strengths:** a good **balance of speed and accuracy** (~59 fps at 300×300); handles **multiple scales** better than early YOLO. - **Weaknesses:** still weaker than two-stage detectors on very small objects; sensitive to anchor design. - **Ideal for:** mobile and embedded devices (with MobileNet backbones), real-time apps that need decent multi-scale accuracy.

	R-CNN (Faster)	YOLO	SSD
stages	two	one	one
speed	slow	fastest	fast
accuracy	highest	good	good
small objects	best	weakest	medium

→ notes: `week-2-3/notes/04_object_detection_rcnn_yolo_ssd.md`

52. Explain techniques used to overcome these challenges: data augmentation, histogram equalization, robust feature detection, and multi-scale feature extraction. 1. **Data augmentation** (for viewpoint, illumination, occlusion and scale): artificially vary the training images so the model learns invariance. - *Viewpoint:* rotations, flips, perspective/affine warps. - *Illumination:* brightness, contrast and color jitter. - *Occlusion:* random erasing / cutout (blank out patches). - *Scale:* random resized crops and zoom. - The model sees many variations of each object without new labels, so it generalises better (AlexNet's crops and flips cut overfitting). 2. **Histogram equalization** (for illumination): spreads intensities over the full range using the CDF, normalising contrast between dark and bright images. **CLAHE** handles uneven lighting locally. Use it before thresholding, feature detection or face recognition. HSV/normalised color spaces help similarly. 3. **Robust feature detection** (for viewpoint, scale, occlusion): use features designed to be invariant. - **SIFT:** scale-space extrema + dominant orientation, so it's scale- and rotation-invariant. - **ORB:** fast, rotation-aware. - **HOG:** block-normalised against lighting. - Because many **local** features are matched (with ratio test + RANSAC), recognition still works when part of the object is **occluded**: enough visible keypoints remain. 4. **Multi-scale feature extraction** (for scale): image pyramids; SSD-style predictions from several feature maps; **Feature Pyramid Networks** combining deep semantic and shallow high-resolution features; anchor boxes of many sizes; Inception's parallel filter sizes. Small and large objects are both detected.

Also: deep CNNs and ViTs trained on large, diverse datasets learn invariance automatically, and attention/context helps reason about occluded parts. → notes: `week-1/notes/day4_histograms_and_thresholding.md`, `week-2/notes/01_feature_detection_harris_sift_orb.md`, `week-2-3/notes/04_object_detection_rcnn_yolo_ssd.md`

53. Discuss the training loop in detail: how labels_real and labels_fake are used, and how gradients flow in each step.

```
for real, _ in loader:
    batch_size = real.size(0)
    labels_real = torch.ones(batch_size, 1)    # target 1 = "real"
    labels_fake = torch.zeros(batch_size, 1)   # target 0 = "fake"

    # ---- Step 1: train D ----
    outputs_real = D(real)
    loss_real = criterion(outputs_real, labels_real)          # real images should
score 1
    noise = torch.randn(batch_size, 100)
    fake = G(noise)
    outputs_fake = D(fake.detach())                            # detach: stop
gradients into G
    loss_fake = criterion(outputs_fake, labels_fake)           # fakes should score 0
    loss_d = loss_real + loss_fake
    optimizer_D.zero_grad(); loss_d.backward(); optimizer_D.step()

    # ---- Step 2: train G ----
    outputs_fake = D(fake)                                     # no detach this time
    loss_g = criterion(outputs_fake, labels_real)              # G wants fakes called
REAL
    optimizer_G.zero_grad(); loss_g.backward(); optimizer_G.step()
```

Labels (BCE loss): - `labels_real` = all ones, `labels_fake` = all zeros, created per batch with `batch_size = real.size(0)` (the last MNIST batch is smaller: 60,000 = 937 × 64 + 32). - **In D's step:** real images get target **1** and fakes target **0**: the truth about where each image came from. BCE = −log D(x) for reals and −log(1 − D(G(z))) for fakes. - **In G's step:** fakes get target **1** (`labels_real`). This isn't cheating: it expresses **G's goal** (to be judged real). The loss −log D(G(z)) is the **non-saturating** generator loss (Q57).

Gradient flow: - **Step 1:** `fake.detach()` cuts the graph, so `loss_d.backward()` computes gradients **only for D's weights**, and `optimizer_D.step()` updates only D. Without detach, gradients would flow into G (wasted computation, and G would wrongly learn to help D). - **Step 2:** `loss_g.backward()` sends gradients **backwards through D and then into G**. D's weights receive gradients too, but they are **not updated**, because only `optimizer_G.step()` is called, and `optimizer_D.zero_grad()` clears them on the next batch. D acts as a fixed **differentiable critic** telling G how to change its pixels to look more real. - **zero_grad()** before each backward call prevents gradients from accumulating across steps.

Settings: Adam, lr = 0.0002 for both ($\beta_1 = 0.5$ is recommended; the default 0.9 caused mode collapse in our run); batch 64; two optimizers, one per network. **Monitoring:** if D outputs 0.5 everywhere, loss_D ≈ 2 ln 2 ≈ 1.386 and loss_G ≈ ln 2 ≈ 0.693. But losses don't show image quality; watch a fixed-noise grid. → notes: `week-4-gans/notes/01_gan_basics.md` , `week-4-gans/notes/02_gan_training_dynamics.md`

54. Elaborate on contour analysis and object tracking: algorithms, challenges (occlusion, scale, clutter), and real-world applications. Contour analysis: - A **contour** is a curve joining continuous boundary points of the same intensity: an object's outline in a binary image. - **Pipeline:** grayscale → blur → threshold/Canny → morphology → `cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`. - **Measurements:** - area (`contourArea`), perimeter (`arcLength`); - centroid from **moments** (cx = M10/M00, cy = M01/M00); - bounding box (`boundingRect`, `minAreaRect`), enclosing circle; - **convex hull** and solidity (area / hull area), aspect ratio, circularity ($4\pi \cdot \text{area}/\text{perimeter}^2$); - shape matching (`matchShapes` with Hu moments). - **Shape classification:** `approxPolyDP` (Douglas–Peucker) simplifies a contour to its corners: 3 vertices = triangle, 4 = rectangle or square (check the aspect ratio), many = circle.

Object tracking: following an object across video frames. - **Centroid / contour tracking:** detect objects each frame (e.g. by background subtraction + contours) and link them by the nearest centroid. - **Mean-Shift / CamShift:** track a color histogram (HSV hue) by moving the window to the peak of the back-projection. CamShift also adapts window size and orientation. - **Optical flow: Lucas–Kanade / KLT** tracks sparse keypoints (Shi–Tomasi corners) assuming constant brightness and small motion, using pyramids for larger motion. **Farneback** gives dense flow for every pixel. - **Kalman filter:** predicts the next position from a motion model and corrects it with measurements. It bridges gaps during occlusion. - **OpenCV trackers:** MOSSE (very fast), KCF, CSRT (accurate). - **Deep trackers:** SORT / DeepSORT (detector + Kalman + appearance features for multiple objects), Siamese networks.

Challenges: - **Occlusion:** the object is partly or fully hidden, so contours break or merge and trackers lose the target or swap identities. *Mitigation:* Kalman prediction, re-identification by appearance, multiple cues. - **Scale change:** the object moves towards or away from the camera, so fixed-size windows fail. *Mitigation:* CamShift, pyramidal optical flow, scale-adaptive trackers (CSRT), multi-scale detection. - **Background clutter:** similar colors or textures nearby confuse color-histogram and contour methods (false contours). *Mitigation:* better segmentation, motion cues, combined features, deep appearance models. - Also: illumination changes (they break the brightness-constancy assumption), fast motion and blur, deformation, and objects entering or leaving the frame.

Applications: traffic monitoring and vehicle counting; surveillance and people counting; sports analytics (ball and player tracking); industrial inspection and object counting on conveyors; gesture and hand tracking (HCI); medical (cell tracking, measuring tumour area); robotics and drones; AR (tracking markers); document scanning (finding the page's four corners). → notes: `week-1/notes/day6_contours_and_shapes.md`, `week-2/notes/03_object_tracking_optical_flow_klt.md`

55. Elaborate on the Bag of Visual Words pipeline: feature extraction, clustering, histogram representation, and classification. Idea: borrowed from text classification (Bag of Words). Represent an image by **how often each "visual word" (a typical local patch pattern) appears**, ignoring where it appears.

1. **Feature extraction:** detect keypoints and compute local **descriptors** for every training image: SIFT (128-D), ORB (binary) or dense SIFT on a grid. Each image yields hundreds or thousands of descriptors, a different number per image.

2. **Clustering, building the visual vocabulary:** pool the descriptors from **training images only** and run **k-means** with k clusters (e.g. 100–1,000). Each cluster centre is a **visual word**: a prototype local pattern, such as a corner type or a texture patch. Like grouping synonyms ("big", "large", "huge" → one word), similar-looking patches map to one word.
3. **Histogram representation, encoding:** for each image, assign every descriptor to its **nearest cluster centre** and count how many times each word occurs, giving a **fixed-length histogram of k bins** whatever the number of keypoints. Normalise it (L1/L2, optionally TF-IDF weighting).
4. **Classification:** train a classifier on the histograms with labels: **SVM** (often with a χ^2 or histogram-intersection kernel), kNN, or Naive Bayes. To classify a new image: extract descriptors → map them to words with the same vocabulary → build its histogram → classifier → label.

Choosing k: too small and different patterns merge (less discriminative); too large and the histograms become sparse and noisy (slower, overfitting). Choose it by validation. **Strengths:** fixed-length vectors from variable feature sets; robust to occlusion and viewpoint (local features); simple; works well for **textures and scenes**. **Weaknesses: loses spatial layout** (fixed by **spatial pyramid matching**); quantization error; the vocabulary depends on the training data; replaced by CNN features in modern systems. **Applications:** texture and material classification, scene recognition, image retrieval (search by similar images), object categorisation, early robot place recognition (loop closure). → notes: `week-2/notes/05_bag_of_visual_words.md`

56. Elaborate on how modern architectures (ResNet, EfficientNet, ViT) solved depth and efficiency challenges, and analyze their impact on real-world vision tasks. The problems after VGG: - **Depth:** deeper plain networks suffered vanishing gradients and the **degradation problem** (a 56-layer plain net had *higher training* error than a 20-layer one). - **Efficiency:** VGG-16 had 138M parameters and ~15 GFLOPs, too heavy for mobile devices. - **Locality:** CNNs see only local neighbourhoods per layer, so global context builds up slowly.

1. ResNet (2015, He et al.): depth. - **Residual blocks** y = F(x) + x: layers learn only the residual, and identity mappings are easy, so extra layers can't hurt. - The gradient flows through the identity shortcut unchanged, so **very deep networks train** (ResNet-50/101/152, even 1,000+ layers). - **Bottleneck blocks** (1×1 → 3×3 → 1×1) keep compute manageable; BatchNorm throughout. - ImageNet top-5 error **3.57%**, below the human estimate (~5%); won ILSVRC 2015. - **Impact:** the default **backbone** for detection (Faster/Mask R-CNN), segmentation and transfer learning; medical diagnosis (X-ray and CT classification); face recognition. Skip connections were adopted everywhere (U-Net, DenseNet, Transformers).

2. EfficientNet (2019, Tan & Le): efficiency. - Earlier networks scaled **one** dimension arbitrarily. **Compound scaling** grows depth (α^φ), width (β^φ) and resolution (γ^φ) together, under the constraint $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$, so each step doubles the compute in a balanced way. - Base network found by **neural architecture search**, built from mobile inverted bottleneck (MBConv) blocks with squeeze-and-excitation. - **EfficientNet-B7** matched the best accuracy (84.3% top-1) with **8.4× fewer parameters** and **6.1× faster** inference than the previous best; B0 is tiny (5.3M parameters). - **Impact:** state-of-the-art accuracy on **phones, edge and embedded devices** (on-device photo classification, agriculture/plant-disease apps, medical screening in clinics); cheaper cloud inference; a backbone for EfficientDet.

3. Vision Transformer, ViT (2020, Dosovitskiy et al.): global context. - An image becomes a **sequence of 16×16 patch tokens** + position embeddings, processed by Transformer encoder layers with **self-attention**: every patch attends to every other patch from the **first layer**, giving **global context** and long-range relationships. - Fewer built-in assumptions (inductive biases) than CNNs, so it needs **large datasets** (JFT-300M) or pre-training. At scale it matches or beats CNNs, and it **scales well** with data and compute. - **Impact:** a unified architecture for vision and language, enabling **multimodal models** (CLIP, image captioning, text-to-image); DETR (Transformer-based detection without anchors or NMS) and Segment Anything; hybrid and efficient variants (Swin Transformer, DeiT).

Analysis: - ResNet made **depth** usable, EfficientNet made accuracy **affordable**, and ViT made **global reasoning** natural. - Together they moved vision from hand-tuned architectures to **scalable, transferable backbones**. - Pre-trained models (ImageNet, CLIP) are fine-tuned for real tasks with little data: medical imaging, autonomous driving (perception stacks with ResNet/EfficientNet backbones and Transformer detectors), retail, satellite imagery, quality inspection. - **Trade-offs remain:** ViTs are data- and compute-hungry; CNNs remain strong on small datasets and edge devices; hybrids combine both.

→ notes: `week-2-3/notes/03_modern_cnns_resnet_efficientnet.md`, `week-2-3/notes/07_attention_and_vision_transformers.md`

57. Analyze the mathematical foundation of GANs: minimax game, optimal Discriminator, Nash equilibrium, and why the non-saturating generator loss is used.
1. The minimax game:

min_G max_D V(D, G) = E_{x~p_data}[log D(x)] + E_{z~p_z}[log(1 − D(G(z)))]

D maximises V (classifies correctly); G minimises V (fools D). Writing p_g for the distribution of G's outputs:

V = ∫ [p_data(x) log D(x) + p_g(x) log(1 − D(x))] dx

2. The optimal Discriminator (for a fixed G): maximise the integrand point by point. For a·log y + b·log(1 − y), the maximum is at y = a/(a + b). So

D*(x) = p_data(x) / (p_data(x) + p_g(x))

Where real data is much more likely than fakes, D* → 1; where they are equally likely, D* = ½.

3. What G minimises: substituting D* gives

V(D*, G) = −log 4 + 2·**JSD(p_data ‖ p_g)**

where JSD is the Jensen–Shannon divergence (≥ 0, and 0 only if the two distributions are identical). So, with an optimal D, training G **minimises the JS divergence** between the real and generated distributions. The global minimum is **p_g = p_data**, with V = −log 4 and D*(x) = ½ everywhere.

4. Nash equilibrium: a point where **neither player can improve by changing only its own strategy**. For GANs that's (G with p_g = p_data, D = ½): G can't get any closer to the data, and D can do no better than guessing. In practice: - training uses **alternating gradient steps** on a non-convex game, so it may **oscillate around** the equilibrium instead of converging; - networks have limited capacity and D is never optimal; - hence instability and mode collapse (Q38). That's why "D ≈ 0.5" alone isn't proof of success: also check sample quality and diversity.

5. Why the non-saturating generator loss? - The original G loss is **minimise log(1 − D(G(z)))**. Early in training G is poor, and D easily rejects fakes: D(G(z)) ≈ 0. - There, log(1 − D) is **flat**. With D = σ(a) (a = D's logit), the gradient of log(1 − σ(a)) with respect to a is −σ(a) ≈ 0. **G gets almost no gradient exactly when it most needs to learn** (saturation). - **Non-saturating loss:** instead **maximise log D(G(z))**, i.e. minimise **−log D(G(z))**. Its gradient with respect to a is −(1 − σ(a)) ≈ −1 when D(G(z)) ≈ 0, so G gets a **strong gradient** when it's doing badly. - Both losses share the same fixed point (they push D(G(z)) towards 1), but the non-saturating one gives much better learning dynamics. - **In code** it's exactly `criterion(D(fake), labels_real)`: BCE with target 1 = −log D(G(z)). That's why the notebook uses `labels_real` for G's step. - It's still not a perfect fix: when D is extremely strong, gradients can become unstable. This motivated **WGAN** (Wasserstein distance, smooth gradients everywhere), label smoothing and spectral normalisation.

Summary: the minimax game → the optimal D is a density ratio → G minimises the JS divergence → the equilibrium is p_g = p_data, D = ½ → in practice alternating SGD only approximates it → the non-saturating loss fixes G's vanishing gradients early in training. → notes: `week-4-gans/notes/01_gan_basics.md`, `week-4-gans/notes/02_gan_training_dynamics.md`